

Expanded Multi-Platform Freelance Job Scraping Engine

& Human-In-The-Loop Integration Architecture

SECTION 1: 16-Platform Data Ingestion Matrix & Analysis

The architectural foundation of the multi-platform freelance job aggregator relies on a highly resilient, distributed data ingestion engine. Extracting structured job post data, client briefs, and project requirements across sixteen distinct platforms requires a highly heterogeneous approach. Platforms range from open, unauthenticated REST APIs to highly fortified, session-locked talent portals defended by enterprise-grade Web Application Firewalls (WAFs). The ingestion nodes must be decoupled from the core application logic, operating autonomously to fetch, normalize, and queue data for downstream AI processing.

The sixteen targeted platforms are categorized by their architectural exposure and accessibility, necessitating bespoke extraction strategies for each to ensure high fidelity and low latency.

The first category involves open application programming interfaces (APIs) and rich site summary (RSS) feeds, which present the lowest barrier to entry and the highest degree of reliability. Freelancer.com operates a robust, publicly accessible REST API that allows for highly specific querying without requiring authenticated session tokens. Job search parameters can be queried directly via the endpoint `https://www.freelancer.com/api/projects/0.1/projects/active/` utilizing standard HTTP GET requests. The platform supports URL-encoded parameters such as `or_search_query` to filter by specific technical keywords, and the payload returns highly structured JSON, entirely eliminating the need for fragile Document Object Model (DOM) parsing. Upwork presents a dual-layered architecture. While the platform utilizes a strict GraphQL API that requires OAuth2 authentication and a specific `X-Upwork-API-TenantId` header for state-modifying actions, it simultaneously exposes public RSS feeds (e.g., `/ab/feed/jobs/rss`) that can be polled for low-latency job discovery without triggering Cloudflare security blocks. Furthermore, the Upwork GraphQL endpoint allows for complex data retrieval using the `marketplaceJobPostings` query combined with the `USER_JOBS_SEARCH` search type, provided the executing API key possesses the "Read marketplace Job Postings - Public" permission. PeoplePerHour operates similarly, allowing job listings to be extracted via public RSS feeds or through direct interception of the platform's internal AJAX and GraphQL requests initiated by the frontend React application. Apify actors are frequently deployed as pre-built microservices to interface with this

platform's specific query structures. Guru also exposes structured RSS feeds categorized by specific industry verticals (e.g., <https://feeder.co/discover/.../guru-com>), which facilitates lightweight, high-frequency polling with minimal infrastructure overhead. Freelancermap supports standard XML and Atom feed readers, making the extraction of open IT contracts highly deterministic and resistant to layout mutations.

The second category encompasses platforms heavily reliant on Single Page Application (SPA) architectures, necessitating Headless Browser DOM Scraping and internal XMLHttpRequest (XHR) interception. Behance (Jobs) dynamically loads project data via infinite scrolling powered by internal XHR JSON APIs rather than static HTML. The data payload intercepted from these background requests includes rich project tags, client tools, and budget indicators, avoiding the high computational cost of rendering the full DOM. Contra is built as a React-based SPA utilizing GraphQL for state management. By deploying headless browsers equipped with network interception middleware, queries such as `GetOpportunities` can be captured and replayed, with location and role parameters injected directly into the GraphQL variable payload. Fiverr presents a significantly higher technical barrier, as it is heavily protected by Datadome and PerimeterX anti-bot solutions. Extraction requires full headless browser emulation utilizing Playwright to bypass initial cryptographic challenges. Once the page successfully executes its JavaScript payload, the Next.js `__NEXT_DATA__` JSON object can be extracted directly from the HTML source script tags, providing the complete initial state of the job postings without further DOM traversal. 99designs operates on a more traditional architecture, requiring programmatic DOM parsing via CSS selectors to extract contest specifications, visual design briefs, and budget tiers, as the platform does not expose a public data API. Legiit and Jobbers.io are structured as standard marketplace applications. Jobbers.io, notable for its 0% commission model, utilizes a straightforward DOM structure that can be parsed rapidly. While basic HTTP clients paired with HTML parsers like Cheerio or BeautifulSoup are sufficient for initial payload delivery, executing headless browsers is recommended to trigger lazy-loaded dynamic elements.

The third category introduces the most substantial engineering challenges: Vetted Talent Portals that are strictly session-locked. Toptal operates as an exclusive, remote-first network protected by aggressive Cloudflare and DataDome implementations, combined with behavioral analysis models. Accessing the internal job board requires an active, vetted talent session. Data extraction must occur via authenticated session injection into a stealth browser, mimicking human interaction patterns to prevent triggering anomaly detection algorithms. Codeable, an expert platform requiring a multi-stage technical vetting process, posts jobs to an internal expert dashboard requiring persistent authentication and complex Cookie and JSON Web Token (JWT) management. Lemon.io and Gun.io similarly operate as highly gated engineering portals. Active developer profiles are required to view the project pipeline, meaning ingestion nodes must dynamically route requests through legitimately established user sessions maintained securely by the backend architecture.

The final category targets Direct Channels and Social Streams. Reddit (r/forhire) is highly accessible via the official Reddit JSON API, accessed by appending `.json` to the subreddit URL. While rate limits apply, authentication is minimal for read-only access, making it a highly reliable source for direct client

listings. Conversely, LinkedIn Public Post Streams are protected by extreme anti-bot measures and sophisticated machine learning models that analyze request cadence and network origins. Extracting public posts requires massive residential proxy networks and advanced TLS fingerprint spoofing to prevent IP bans and account flagging, often making client-side extraction via browser extensions a safer alternative to backend scraping.

Platform	Extraction Path	Authentication Level	Anti-Bot / Security Defense	Discovery Latency	Fragility Score
Upwork	GraphQL API / RSS	OAuth2 / Public RSS	Cloudflare, TLS Fingerprinting	1-5 mins	Medium
Freelancer.com	REST API	Public / Guest Token	Cloudflare (Web), Rate Limiting (API)	< 1 min	Low
Fiverr	Next.js JSON Extraction	Public / Session Cookies	Datadome, PerimeterX, CAPTCHA	5-15 mins	High
PeoplePerHour	XHR Intercept / RSS	Public	Basic IP Rate Limiting	5 mins	Low
Guru	RSS / DOM Parsing	Public	Akamai, Rate Limiting	5 mins	Low
Contra	GraphQL Interception	Public	Cloudflare	10 mins	Medium
Toptal	Authenticated DOM/XHR	Vetted Session	Cloudflare, DataDome, Behav. Analysis	15-30 mins	Very High
Lemon.io	Authenticated XHR	Vetted Session	Custom Session Tracking	15-30 mins	High
Gun.io	Authenticated XHR	Vetted Session	JWT Expiration, Rate Limiting	15-30 mins	High
Codeable	Authenticated API	Vetted Session	Cloudflare, Strict Auth	15 mins	High
99designs	DOM Parsing	Public	Fastly, Basic Fingerprinting	10 mins	Medium
Behance	Internal XHR JSON	Public	Akamai	5 mins	Low
Freelancermap	RSS / API	Public	Basic Rate Limiting	< 5 mins	Low

Platform	Extraction Path	Authentication Level	Anti-Bot / Security Defense	Discovery Latency	Fragility Score
Legiit	DOM Parsing	Public	Cloudflare (Standard)	10 mins	Low
Jobbers.io	DOM Parsing	Public	Basic Rate Limiting	10 mins	Low
Reddit	JSON API	Public / App Token	API Rate Limiting	< 1 min	Low
LinkedIn	HTML / XHR Intercept	Guest Tokens / Auth	Cloudflare, Datadome, ML Models	15-60 mins	Extreme

Regardless of the extraction method, the ingestion nodes must map platform-specific payloads to a standardized extraction schema. The required target keys involve complex CSS selectors or JSON path queries depending on the extraction strategy.

Target Data Attribute	Extraction Schema Strategy	Platform-Specific Example
Title & Description	DOM <code>div.job-title</code> , JSON description keys, or markdown body parsing.	Fiverr: <code>__NEXT_DATA__.props.pageProps.gig.title</code>
Budget/ Hourly Rate	JSON extraction of nested currency and minimum/ maximum constraints.	Freelancer.com: <code>budget.minimum</code> , <code>budget.maximum</code> , <code>currency</code>
Skills Required	Array mapping from embedded tags or GraphQL response nodes.	Upwork: <code>ontologySkills</code> array within GraphQL payload
Client Rating/ Country	Extraction from nested poster metadata blocks.	Freelancer.com: <code>poster.country</code> , <code>poster.displayName</code>
Direct Job URL	String concatenation of base URI and canonical platform IDs.	Reddit: <code>https://reddit.com</code> + permalink
Posting Timestamp	Parsing ISO-8601 strings to calculate job freshness and queue priority.	Toptal/Freelancer: <code>postedAt</code> UNIX timestamp conversion

SECTION 2: Anti-Detection, Proxy, and Scraping Infrastructure

To sustain enterprise-grade data ingestion across platforms exhibiting high fragility scores, the architecture must implement a multi-layered evasion and infrastructure strategy. Standard HTTP clients—such as Python's `requests` or Node.js `axios`—face immediate termination from advanced WAFs like DataDome and Akamai. These firewalls analyze the cryptographic and behavioral fingerprints of incoming requests, necessitating a highly sophisticated spoofing framework.

The Proxy Management Framework serves as the primary defense against rate limiting and IP reputation blacklisting. Traffic must be dynamically routed through specialized proxy pools based on the target platform's specific defense mechanisms. Datacenter Proxies, which offer high-speed, cost-effective routing, are deployed for unauthenticated API endpoints such as the Freelancer.com API, Upwork RSS feeds, and Reddit. Because these endpoints rely primarily on volumetric rate limiting rather than deep behavioral analysis, continuously rotating datacenter IPs provides sufficient obfuscation. Residential Proxies are mandated for platforms like Fiverr, Contra, Behance, and LinkedIn. These proxies route traffic through standard Internet Service Providers (ISPs), ensuring the requests appear to originate from legitimate consumer devices. For platforms relying on session cookies or guest tokens, the proxy manager must implement sticky IP sessions, maintaining the same IP address for a duration of 10 to 30 minutes. Rapid, uncoordinated IP rotation triggers geolocation anomaly alerts within the WAF, immediately invalidating the session. Mobile Proxies representing 4G and 5G cellular networks sit behind Carrier-Grade NAT (CGNAT). WAFs rarely ban mobile IPs because doing so would simultaneously block thousands of legitimate users sharing the same external IP address. Consequently, mobile proxies are reserved exclusively for Tier 3 platforms like Toptal, Lemon.io, and Codeable, ensuring that the highly valuable vetted talent sessions remain insulated from network-level bans.

When headless browsers are deployed via Playwright or Puppeteer, default configurations broadcast specific cryptographic signatures and DOM markers indicating automated control. The evasion architecture relies on manipulating the Chrome DevTools Protocol (CDP) to strip these markers. Middleware such as `puppeteer-extra-plugin-stealth` must be configured to delete the `navigator.webdriver` property, override the `window.chrome` object, and spoof the Permissions API to perfectly mimic a standard consumer browser. Furthermore, WebGL and Canvas fingerprinting must be obfuscated. WAFs execute silent JavaScript payloads that render hidden graphics and calculate a cryptographic hash of the output to uniquely identify the hardware environment. The evasion framework intercepts these API calls and injects subtle, mathematically consistent noise into the graphic rendering pipeline, ensuring the browser appears as a unique, legitimate hardware configuration that changes across sessions but remains internally consistent during a single session.

At the network transport layer, TLS and HTTP/2 Fingerprinting present the most formidable anti-bot mechanisms. Security providers like DataDome analyze the exact sequence of cryptographic ciphers exchanged during the TLS handshake, commonly referred to as JA3/JA4 fingerprinting. A standard Node.js or Python environment utilizes OpenSSL, which produces a drastically different handshake signature than the NSS or BoringSSL implementations used by Chrome or Safari. To bypass this, the

architecture routes non-browser API requests through specialized HTTP clients engineered to spoof browser TLS signatures, such as `curl-impersonate` or `got-scraping`. Crucially, the HTTP User-Agent header must perfectly synchronize with the TLS fingerprint, the Accept-Language headers, and the underlying operating system specified in the HTTP/2 frame settings (such as TCP window sizes and pseudo-header ordering). Any mismatch—such as presenting a Windows Chrome User-Agent while executing a Linux OpenSSL TLS handshake—results in immediate, unrecoverable blocking.

Handling authenticated sessions for vetted portals requires an air-gapped approach to credential management. For platforms like Toptal and Codeable, where scraping requires active credentials belonging to real developers, a ban represents a severe operational failure. Authentication sequences are never executed by the automated scrapers. Instead, login challenges are solved manually or via a secure Human-in-the-Loop (HITL) relay. The resulting session Cookies, JSON Web Tokens (JWTs), and LocalStorage states are encrypted and stored in a secure Redis vault. When a scheduled ingestion job initiates, the headless browser is launched on a high-trust Mobile Proxy. The vault injects the session state directly into the browser context via CDP before navigating to the target URL. To further evade detection, behavioral pacing is enforced; the scraper executes strictly within human operational limits. Clicks, scrolls, and DOM mutations are injected with randomized Bezier-curve mouse movements and variable latency, enforcing pauses of 2,000 to 5,000 milliseconds between page navigations to simulate natural cognitive processing times.

SECTION 3: Unified Data Normalization & Deduplication Engine

Once data is successfully extracted, the disparate payload structures from sixteen different platforms must be transformed, unified, and cleansed. The normalization microservice ingests raw JSON and HTML payloads and maps them to a strictly typed, version-controlled schema before passing the data to the AI proposal generation pipeline.

The `UnifiedJobPosting` schema must accommodate the extreme variance in freelance contracting models, ranging from hourly development work to fixed-price design contests and retainer-based consulting.

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "UnifiedJobPosting",
  "type": "object",
  "required": ["id", "platform", "source_url", "title", "description", "posted_at",
"status"],
  "properties": {
    "id": { "type": "string", "description": "Global Unique Identifier (UUIDv4)" },
    "platform": { "type": "string", "enum": ["upwork", "freelancer", "toptal",
"codeable", "contra", "99designs", "reddit", "linkedin" ] },
    "source_id": { "type": "string", "description": "Platform-specific job ID for
state tracking" },
    "source_url": { "type": "string", "format": "uri" },
    "title": { "type": "string" },
    "description": { "type": "string", "description": "Markdown formatted
description" },
    "budget": {
      "type": "object",
      "properties": {
        "type": { "type": "string", "enum": ["fixed", "hourly", "contest",
"unspecified" ] },
        "currency": { "type": "string", "maxLength": 3 },
        "min_amount": { "type": "number" },
        "max_amount": { "type": "number" }
      }
    },
    "skills": {
      "type": "array",
      "items": { "type": "string" }
    },
    "client": {
      "type": "object",
      "properties": {
        "client_id": { "type": "string" },
        "country": { "type": "string" },
        "rating": { "type": "number" },
        "history_spend": { "type": "number" }
      }
    },
    "posted_at": { "type": "string", "format": "date-time" },
    "ingested_at": { "type": "string", "format": "date-time" },
    "status": { "type": "string", "enum": ["active", "closed", "drafted" ] }
  }
}

```

A significant challenge in multi-platform aggregation is the high frequency of cross-posting. Clients routinely post the exact same freelance brief across multiple platforms—such as Upwork and Freelancer.com—to maximize talent visibility. To prevent the AI from generating redundant proposals for the identical client, a highly accurate deduplication engine is deployed at the ingestion layer. The system first evaluates hard heuristics. If the exact same title string, client country, and budget range appear within a 48-hour window across two platforms, the records are flagged as potential duplicates. However, clients frequently alter project titles or rephrase descriptions slightly when switching platforms. Therefore, a Semantic Vector Embedding pipeline is required. The text of the title and description is processed through a lightweight, high-performance Natural Language Processing (NLP) model, such as `all-MiniLM-L6-v2`. The resulting high-dimensional vector embeddings are stored alongside the relational data. As new jobs enter the pipeline, their embeddings are mathematically compared against the vectors of all jobs ingested over the trailing 72 hours. By calculating the cosine similarity between the vectors, the system can determine semantic equivalence. A cosine similarity score exceeding a threshold of 0.92 automatically links the postings into a single `JobCluster` entity. This ensures the dashboard displays the project only once, allowing the freelancer to selectively choose the most advantageous platform (e.g., the one with the lowest commission fees) on which to submit the proposal.

The storage architecture is designed to handle immense throughput while providing rapid querying capabilities for the frontend dashboard. PostgreSQL serves as the primary operational database, handling ACID-compliant storage of normalized jobs. The `UnifiedJobPosting` payload is stored in a JSONB column, which provides the flexibility of a NoSQL document store within a relational framework. This data is indexed via Generalized Inverted Indexes (GIN), allowing for rapid, schema-less querying of nested keys (for example, querying all jobs where `budget.currency` is USD and `client.rating` is greater than 4.5). To support the semantic deduplication logic, the `pgvector` extension is integrated directly into the PostgreSQL cluster, enabling native execution of cosine similarity queries (`<=>`) against the stored embeddings without requiring a separate vector database infrastructure. MongoDB is utilized strictly as a cold-storage data lake, archiving the raw, un-normalized HTML and JSON payloads extracted from the 16 platforms. This allows data engineers to replay historical data if the normalization schema requires retroactive adjustments. Finally, Redis operates as an in-memory publish/subscribe message broker. As jobs are scraped, they are pushed to Redis queues managed by BullMQ. Worker nodes consume these queues asynchronously, handling the CPU-intensive tasks of normalization, vector embedding, and AI proposal generation, ensuring the scraping nodes are never blocked by database write latency.

SECTION 4: Terms of Service (TOS), Compliance & Account Safety

Operating an aggressive multi-platform ingestion engine carries significant legal, operational, and ethical liabilities. The architecture relies on strict physical and logical boundaries to ensure legal compliance and the absolute safety of the end-user's platform accounts.

Scraping public data operates within specific, yet continuously evolving, legal frameworks globally. In the United States, the landmark *hiQ Labs v. LinkedIn* ruling established a substantial legal precedent regarding the Computer Fraud and Abuse Act (CFAA). The courts affirmed that scraping publicly accessible data on the internet does not generally violate the CFAA, provided no protective authentication barriers are circumvented without authorization. For operations involving Indian jurisdictions, the Digital Personal Data Protection Act, 2023 (DPDPA) provides a critical exemption. Section 3(c)(ii) of the DPDPA 2023 explicitly states that the regulatory framework does not apply to personal data made publicly available by the Data Principal or under a legal obligation. Consequently, extracting publicly available client names, project histories, or freelancer profiles does not trigger DPDPA compliance penalties, provided the data is unequivocally public. In the European Union, the Digital Services Act (DSA) promotes transparency, and scraping for market research and lead generation remains viable, though scraping private or restricted data without explicit permission remains a severe violation of both the General Data Protection Regulation (GDPR) and established ethical guidelines. The architectural directive dictates that Tier 1 and Tier 2 platforms are treated as public data repositories, while extraction from Tier 3 vetted portals relies entirely on authorized user consent to access their own dashboard data.

To completely eliminate the risk of the end-user's platform accounts being terminated by automated security systems, the system employs a strict decoupling architecture, known as the Air-Gap Principle. The server-side scraping infrastructure never utilizes the end-user's actual credentials. Public platforms are scraped entirely unauthenticated or via synthetic burner accounts provisioned exclusively for data extraction. The backend ingestion engine operates in total isolation from the user's identity.

When the backend AI pipeline generates a tailored proposal based on the scraped data, this proposal is transmitted via WebSockets to a Google Chrome Extension (built on Manifest V3) installed on the end-user's local machine. The extension operates entirely within the user's legitimate, organically authenticated browser session. This defines the Safe Automation Limits required to remain compliant with platform Terms of Service. Total automation of proposal submission—such as using an API to automatically send a bid without human oversight—violates the operational policies of virtually all freelance platforms and frequently triggers immediate algorithmic bans. Therefore, automated pitch submission is strictly prohibited at the architectural level. Instead, the system mandates a Human-In-The-Loop (HITL) paradigm. The Chrome Extension acts purely as an advanced auto-fill utility. The user navigates to the job posting natively within their browser, reviews the AI-generated text presented by the extension, and approves it. The extension then injects the text into the DOM utilizing standard JavaScript manipulations (e.g., `element.value = text` or a one-click clipboard launch). Crucially, the extension must never programmatically click the "Submit Bid" button. The human user must execute the final physical action, transforming the workflow from prohibited bot automation into compliant AI-assisted drafting.

SECTION 5: Implementation Roadmap & Technical Stack

The technical implementation requires orchestrating multiple distinct engineering disciplines, from network manipulation and proxy routing to natural language processing and browser extension development.

5.1 End-to-End System Architecture Diagram

```

graph TD
    subgraph Scraping_Nodes [Distributed Ingestion Engine]
        A1[Tier 1: API / RSS Scrapers]
        A2[Tier 2: DOM / XHR Interceptors]
        A3[Tier 3: Vetted Stealth Browsers]
    end

    subgraph Network_Layer
        P1[Datacenter Proxies]
        P2[Residential Sticky Proxies]
        P3[Mobile CGNAT Proxies]
    end

    A1 --> P1
    A2 --> P2
    A3 --> P3

    subgraph Message_Broker
        B[Redis / BullMQ - Task Queues]
    end

    subgraph Data_Pipeline
        C[Normalization Microservice]
        D[Semantic Vector Embedder]
        E[(PostgreSQL JSONB + pgvector)]
        M[(MongoDB Raw Data Lake)]
    end

    subgraph AI_Delivery [AI & Delivery]
        F[LLM Proposal Generator]
        G[Dashboard Web App]
        H[Manifest V3 Chrome Extension]
    end

    P1 -->|Raw JSON/XML| B
    P2 -->|Raw DOM/XHR| B
    P3 -->|Authenticated Data| B

    B --> C
    C -->|Store Raw| M
    C --> D
    D <--> E
    D -->|New Unique Job Event| F
    F -->|Drafted Proposal| E

    E --> G

```

5.2 Tech Stack Matrix

Architectural Component	Recommended Technology / Framework	Engineering Justification
Backend API & Routing	Node.js (TypeScript)	Provides high concurrency for non-blocking network I/O, essential for managing thousands of simultaneous HTTP requests.
DOM Scraping Engine	Playwright + Stealth Plugin	Emulates full browser contexts with CDPR manipulation to bypass Datadome and PerimeterX JavaScript challenges.
HTTP Request Client	got-scraping / curl-impersonate	Enables precise spoofing of JA3/JA4 TLS fingerprints and HTTP/2 pseudo-headers for non-headless API calls.
Queue & Cache Broker	Redis + BullMQ	Handles rapid ingestion bursts and coordinates distributed scraper nodes with guaranteed delivery mechanisms.
Primary Database	PostgreSQL + pgvector	Robust JSONB storage for varied payloads and native vector search for semantic deduplication without external dependencies.
Data Lake Archive	MongoDB	Highly scalable cold-storage for archiving raw, un-normalized HTML/JSON payloads for debugging and replay capability.
AI Integration Layer	OpenAI (GPT-4o) / Anthropic	Analyzes complex job constraints and generates highly contextual, persuasive proposal drafts tailored to the platform.
Frontend UI	React / Next.js	Delivers a real-time, WebSocket-enabled dashboard for users to review incoming, deduplicated job streams.
HITL Delivery Agent	Chrome Extension (Manifest V3)	Securely interfaces with the user's local DOM for safe proposal injection using Service Workers and Content Scripts.

5.3 Phased Implementation Plan

To mitigate overarching engineering risks and achieve a rapid Return on Investment (ROI), the system architecture is deployed in four progressive phases, advancing from the most accessible platforms to the most defensively fortified.

Phase 1: Minimum Viable Product & Foundation (Tier 1 Platforms)

The initial phase focuses on establishing the core infrastructure and ingesting data from the most accessible platforms. The engineering team will deploy the PostgreSQL, MongoDB, and Redis

databases. The `UnifiedJobPosting` JSON schema will be finalized. Scraping modules will be built targeting Upwork (via RSS and public API endpoints), Freelancer.com (via its open REST API), PeoplePerHour, and Guru. Datacenter proxies will be configured for rate limit evasion. By the end of this four-week phase, the baseline AI drafting pipeline and a rudimentary web dashboard will be operational, allowing users to view ingested jobs and generated proposals.

Phase 2: DOM Evasion & SPA Interception (Tier 2 Platforms)

With the foundation secure, the focus shifts to platforms relying on complex frontend architectures. The Playwright headless browser infrastructure will be deployed, integrated with the proxy manager to utilize residential IPs with sticky session capabilities. The engineering team will reverse-engineer the Next.js and React GraphQL states utilized by Fiverr, Contra, Behance, Legiit, Jobbers.io, and 99designs. This enables the extraction of data payloads directly from XHR intercepts rather than computationally expensive DOM rendering. This phase, spanning weeks five through eight, introduces the `puppeteer-extra-plugin-stealth` modifications to bypass mid-tier WAF deployments.

Phase 3: High-Security Authentication & Vector Search (Tier 3 & 4 Platforms)

Phase three tackles the most difficult ingestion targets: Toptal, Codeable, Lemon.io, Gun.io, and the direct social channels of LinkedIn and Reddit. The secure Redis vault for encrypted credential and session management will be implemented. The system will deploy Mobile Proxy routing to ensure that the vetted platform sessions are never associated with datacenter IPs. Concurrently, the vector-based semantic deduplication engine (utilizing `all-MiniLM-L6-v2` and `pgvector`) will be finalized to handle the massive influx of overlapping data across all integrated platforms. This highly complex phase will require five weeks of iterative development to ensure zero account compromise.

Phase 4: Human-in-the-Loop Integration & Production Scale

The final implementation phase focuses on user delivery and compliance. The Manifest V3 Chrome Extension will be finalized and deployed to the Chrome Web Store (or distributed internally). Secure WebSocket connections will be established between the extension's service worker and the backend dashboard to push approved AI drafts directly to the user's browser. Robust DOM injection scripts will be written, tailored specifically to the unique user interfaces of each of the sixteen platforms. Finally, comprehensive compliance auditing will be executed to guarantee the air-gap separation of backend scraping accounts from the end-user's organic platform accounts, ensuring strict adherence to platform Terms of Service prior to full production scale.